

Module End Assignment:

“Analyzing E-Learning Platform Purchases using MySQL”

1. Create the database and schema. Populate the Schema:

Created a Database and all three tables in MySQL with appropriate **data types** and **relationships**.

The screenshot displays the MySQL Workbench interface. On the left, the 'SCHEMAS' pane shows a tree view with 'learning' expanded, containing 'Tables' (courses, learners, purchase), 'Views', 'Stored Procedures', and 'Functions'. The 'purchase' table is selected, and its structure is shown in the 'Table: purchase' pane: purchase_id (int PK), learners_id (int), course_id (int), quantity (int), and purchase_date (date). The main editor shows the SQL script for creating the database and tables. The script is as follows:

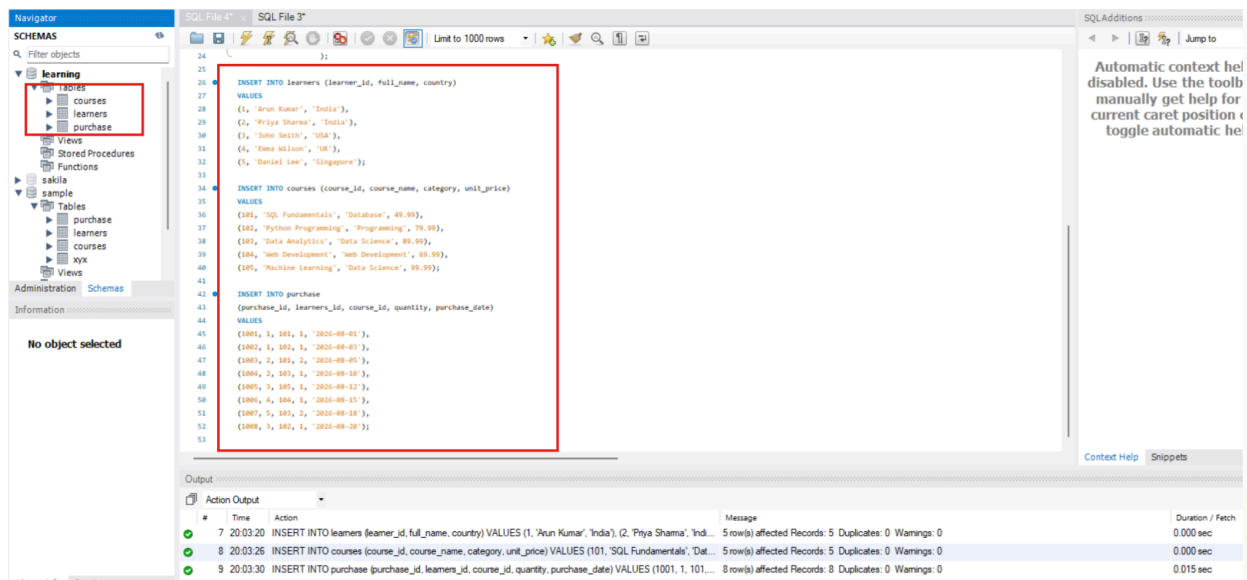
```
1 CREATE DATABASE Learning;
2 USE Learning;
3 CREATE TABLE learners(
4     learner_id INT PRIMARY KEY,
5     full_name VARCHAR(100),
6     country VARCHAR(100)
7 );
8
9 CREATE TABLE courses(
10     course_id INT PRIMARY KEY,
11     course_name VARCHAR(100),
12     category VARCHAR(100),
13     unit_price DECIMAL(10,2)
14 );
15
16 CREATE TABLE purchase(
17     purchase_id INT PRIMARY KEY,
18     learners_id INT,
19     course_id INT,
20     quantity INT,
21     purchase_date DATE,
22     FOREIGN KEY (learners_id) REFERENCES learners(learner_id),
23     FOREIGN KEY (course_id) REFERENCES courses(course_id)
```

The 'Output' pane at the bottom shows the execution results:

#	Time	Action	Message
4	08:16:32	CREATE TABLE courses(course_id INT PRIMARY KEY, course_name VARCHAR(100),	... 0 row(s) affected
5	08:25:02	CREATE TABLE courses(course_id INT PRIMARY KEY, course_name VARCHAR(100),	... Error Code: 1050. Table 'courses' already exists
6	08:25:17	CREATE TABLE purchase(purchase_id INT PRIMARY KEY, learners_id INT, course...	... 0 row(s) affected

Inserted sample data covering:

- **5 learners**
- **5 courses** across multiple categories
- **8 purchase records**



2. Data Exploration Using Joins

Data Presentation Guidelines for the following query

- Formatted currency values to **2 decimal places** where applicable.
- Used meaningful column aliases such as AS total_revenue and AS total_spent.
- Sorted query results appropriately, including sorting by **highest total spending first** where required.
- Combined learner, course, and purchase data using **LEFT JOINS** to display each learner's purchase details, including **course name, category, quantity, total amount, and purchase date**.

SQL File 4* SQL File 3*

Display each learner's purchase details (course name, category, quantity, total amount, and purchase date).*/

```
71
72
73 SELECT l.full_name,
74       c.course_name,
75       c.category,
76       p.quantity,
77       FORMAT(p.quantity*c.unit_price,2) AS total_revenue,
78       p.purchase_date
79 FROM learners l
80 LEFT JOIN purchase p
81     on l.learner_id = p.learners_id
82 LEFT JOIN courses c
83     on c.course_id= p.course_id
84 ORDER BY total_revenue DESC;
```

Result Grid

full_name	course_name	category	quantity	total_revenue	purchase_date
John Smith	Machine Learning	Data Science	1	99.99	2026-08-12
Priya Sharma	SQL Fundamentals	Database	2	99.98	2026-08-05
Priya Sharma	Data Analytics	Data Science	1	89.99	2026-08-10
Arun Kumar	Python Programming	Programming	1	79.99	2026-08-03
John Smith	Python Programming	Programming	1	79.99	2026-08-20

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	20:35:02	SELECT l.full_name, c.course_name, c.category, p.quantity, FORMAT(p.quantity*c.unit_price,2) AS total_revenue...	8 row(s) returned	0.000 sec / 0.000 sec

Combined learner, course, and purchase data using **INNER JOINS** to display each learner's purchase details, including **course name, category, quantity, total amount, and purchase date**

SQL File 4* SQL File 3*

ORDER BY total_revenue DESC;

```
84
85
86 SELECT l.full_name,
87       c.course_name,
88       c.category,
89       p.quantity,
90       FORMAT(p.quantity*c.unit_price,2) AS total_revenue,
91       p.purchase_date
92 FROM learners l
93 INNER JOIN purchase p
94     on l.learner_id = p.learners_id
95 INNER JOIN courses c
96     on c.course_id= p.course_id
97 ORDER BY total_revenue DESC;
```

Result Grid

full_name	course_name	category	quantity	total_revenue	purchase_date
John Smith	Machine Learning	Data Science	1	99.99	2026-08-12
Priya Sharma	SQL Fundamentals	Database	2	99.98	2026-08-05
Priya Sharma	Data Analytics	Data Science	1	89.99	2026-08-10
Arun Kumar	Python Programming	Programming	1	79.99	2026-08-03
John Smith	Python Programming	Programming	1	79.99	2026-08-20

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	20:35:02	SELECT l.full_name, c.course_name, c.category, p.quantity, FORMAT(p.quantity*c.unit_price,2) AS total_revenue...	8 row(s) returned	0.000 sec / 0.000 sec
2	20:38:49	SELECT l.full_name, c.course_name, c.category, p.quantity, FORMAT(p.quantity*c.unit_price,2) AS total_revenue...	8 row(s) returned	0.047 sec / 0.000 sec

Combined learner, course, and purchase data using **RIGHT JOINS** to display each learner's purchase details, including **course name, category, quantity, total amount, and purchase date**

The screenshot shows a SQL IDE interface with a query editor and a result grid. The query is as follows:

```

SELECT l.full_name,
       c.course_name,
       c.category,
       p.quantity,
       FORMAT(p.quantity*c.unit_price,2) AS total_revenue,
       p.purchase_date
FROM learners l
RIGHT JOIN purchase p
  ON l.learner_id = p.learners_id
RIGHT JOIN courses c
  ON c.course_id= p.course_id
ORDER BY total_revenue DESC;

```

The result grid displays the following data:

full_name	course_name	category	quantity	total_revenue	purchase_date
John Smith	Machine Learning	Data Science	1	99.99	2026-08-12
Priya Sharma	SQL Fundamentals	Database	2	99.98	2026-08-05
Priya Sharma	Data Analytics	Data Science	1	89.99	2026-08-10
Arun Kumar	Python Programming	Programming	1	79.99	2026-08-03
John Smith	Python Programming	Programming	1	79.99	2026-08-20

3. Analytical Queries

Q1. Calculated and displayed each learner's **total spending** using $\text{quantity} \times \text{unit_price}$, along with their **country**, by combining learner and purchase data and grouping the results by learner and country.

The screenshot shows a SQL IDE interface with an analytical query and its result grid. The query is as follows:

```

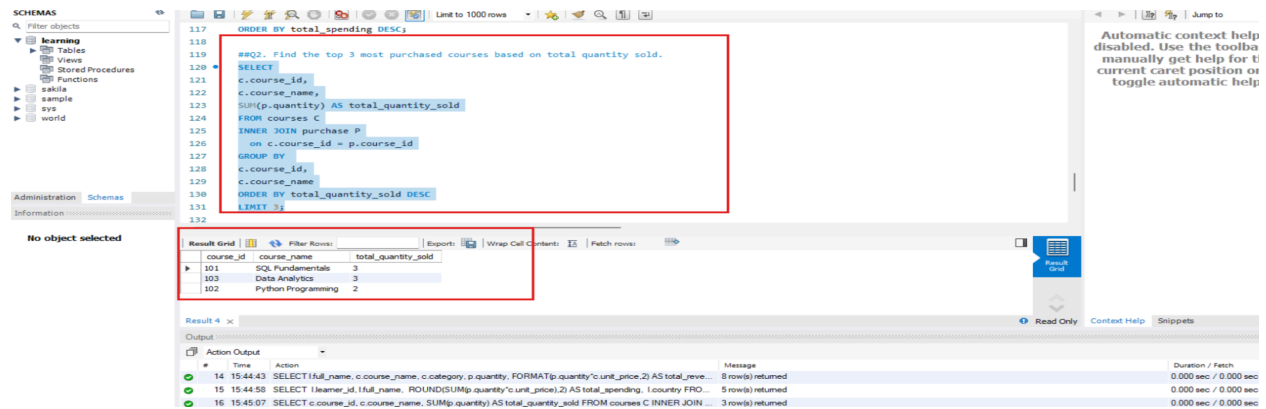
SELECT
  l.learner_id,
  l.full_name,
  SUM(p.quantity*c.unit_price) AS total_spending,
  c.country
FROM learners l
LEFT JOIN purchase p
  ON l.learner_id = p.learners_id
LEFT JOIN courses c
  ON p.course_id = c.course_id
GROUP BY
  l.learner_id,
  l.full_name,
  c.country
ORDER BY total_spending DESC;

```

The result grid displays the following data:

learner_id	full_name	total_spending	country
2	Priya Sharma	199.97	India
3	John Smith	179.98	USA
5	Daniel Lee	179.98	Singapore
1	Arun Kumar	129.98	India
4	Emma Wilson	69.99	UK

Q2. Identified the top 3 most purchased courses by calculating the total quantity sold for each course, grouping the results by course, sorting in descending order of total quantity, and limiting the output to the top 3 courses.



The screenshot shows a SQL IDE interface with a query editor on the left and a results pane on the right. The query editor contains the following SQL code:

```

117 ORDER BY total_spending DESC;
118
119 --Q2. Find the top 3 most purchased courses based on total quantity sold.
120 SELECT
121   c.course_id,
122   c.course_name,
123   SUM(p.quantity) AS total_quantity_sold
124 FROM courses C
125 INNER JOIN purchase P
126   ON c.course_id = p.course_id
127 GROUP BY
128   c.course_id,
129   c.course_name
130 ORDER BY total_quantity_sold DESC
131 LIMIT 3;
132

```

The results pane shows the following data:

course_id	course_name	total_quantity_sold
101	SQL Fundamentals	3
103	Data Analytics	3
102	Python Programming	2

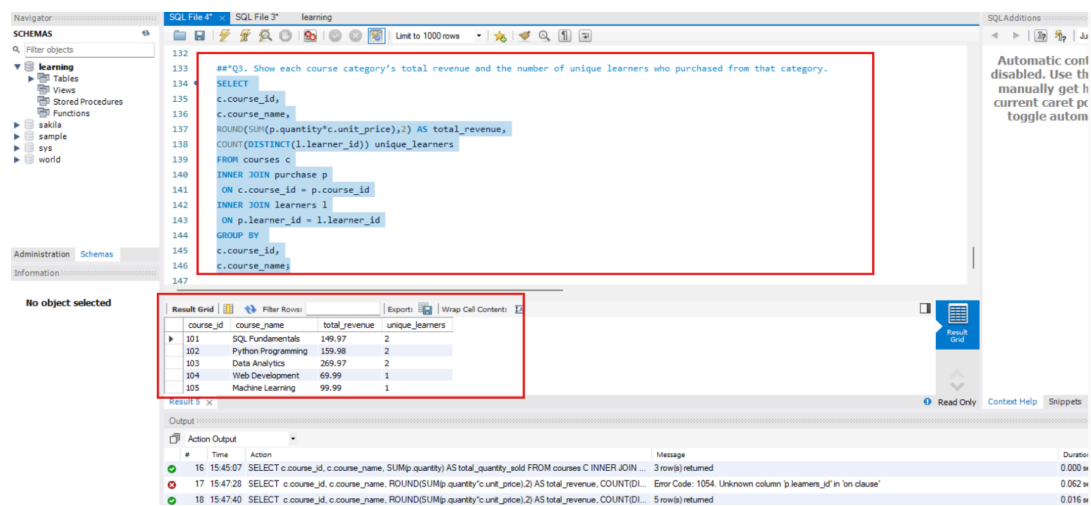
The output pane shows the execution log with the following messages:

```

14 15:44:43 SELECT l.full_name, c.course_name, c.category, p.quantity, FORMAT(p.quantity*c.unit_price,2) AS total_reve... 8 row(s) returned
15 15:44:58 SELECT (learner_id,l.full_name, ROUND(SUM(p.quantity*c.unit_price),2) AS total_spending, 1 country FRO... 5 row(s) returned
16 15:45:07 SELECT c.course_id, c.course_name, SUM(p.quantity) AS total_quantity_sold FROM courses C INNER JOIN ... 3 row(s) returned

```

Q3. Identified learners who purchased courses from more than one category by joining the learners, purchase, and courses tables, counting distinct course categories for each learner, and filtering the results using HAVING category_count > 1



The screenshot shows a SQL IDE interface with a query editor on the left and a results pane on the right. The query editor contains the following SQL code:

```

132
133 --Q3. Show each course category's total revenue and the number of unique learners who purchased from that category.
134 SELECT
135   c.course_id,
136   c.course_name,
137   ROUND(SUM(p.quantity*c.unit_price),2) AS total_revenue,
138   COUNT(DISTINCT(l.learner_id)) unique_learners
139 FROM courses c
140 INNER JOIN purchase p
141   ON c.course_id = p.course_id
142 INNER JOIN learners l
143   ON p.learner_id = l.learner_id
144 GROUP BY
145   c.course_id,
146   c.course_name
147

```

The results pane shows the following data:

course_id	course_name	total_revenue	unique_learners
101	SQL Fundamentals	149.97	2
102	Python Programming	159.98	2
103	Data Analytics	269.97	2
104	Web Development	69.99	1
105	Machine Learning	99.99	1

The output pane shows the execution log with the following messages:

```

16 15:45:07 SELECT c.course_id, c.course_name, SUM(p.quantity) AS total_quantity_sold FROM courses C INNER JOIN ... 3 row(s) returned
17 15:47:28 SELECT c.course_id, c.course_name, ROUND(SUM(p.quantity*c.unit_price),2) AS total_revenue, COUNT(DI... Error Code: 1054. Unknown column 'p.learner_id' in 'on clause'
18 15:47:40 SELECT c.course_id, c.course_name, ROUND(SUM(p.quantity*c.unit_price),2) AS total_revenue, COUNT(DI... 5 row(s) returned

```

Q4. Identified learners who purchased courses from more than one category by joining the learners, purchase, and courses tables, counting distinct course categories for each learner, and filtering the results using HAVING category_count > 1.

The screenshot shows a SQL IDE interface with a query editor and a results grid. The query editor contains the following SQL code:

```

SELECT
  l.learner_id,
  l.full_name,
  count(distinct c.category) AS category_count
FROM learners l
INNER JOIN purchase p
  ON l.learner_id = p.learner_id
INNER JOIN courses c
  ON p.course_id = c.course_id
GROUP BY
  l.learner_id,
  l.full_name
HAVING category_count > 1;

```

The results grid shows the following data:

learner_id	full_name	category_count
1	Arun Kumar	2
2	Priya Sharma	2
3	John Smith	2

The output pane shows the execution of the query, indicating that 3 rows were returned.

Q5. Identified courses that have not been purchased at all by using a LEFT JOIN between the courses and purchase tables and filtering for records where purchase_id IS NULL.

The screenshot shows a SQL IDE interface with a query editor and a results grid. The query editor contains the following SQL code:

```

GROUP BY
  l.learner_id,
  l.full_name
HAVING category_count > 1;

-- Q.5 Identify courses that have not been purchased at all.
SELECT
  c.course_id,
  c.course_name
FROM courses c
LEFT JOIN purchase p
  ON c.course_id = p.course_id
WHERE p.purchase_id IS NULL;

```

The results grid shows the following data:

course_id	course_name
1	Course 1
2	Course 2
3	Course 3

The output pane shows the execution of the query, indicating that 3 rows were returned.

